

Reinforcement Learning

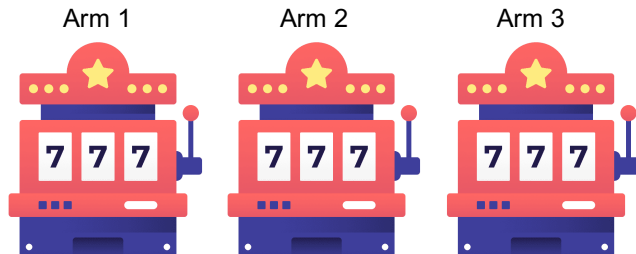
Multi-armed Bandits

Min-hwan Oh

Seoul National University

Multi-armed Bandits

- Multiple slot machines to choose from
- Simplification from RL: actions do not have delayed effect nor affect the state transition



Multi-armed Bandits

Question: given the previous sequence of rewards, which arm would you take next?

- Arm 1: 1, 2, 0, 3, 1, 1, 2, 0
- Arm 2: 50, 30, 0, 0, 0, 0, 0, 0

Multi-armed Bandits

Question: given the previous sequence of rewards, which arm would you take next?

- Arm 1: 1, 2, 0, 3, 1, 1, 2, 0 Running avg. = 1.25
- Arm 2: 50, 30, 0, 0, 0, 0, 0, 0 Running avg. = 10.0

k -armed Bandit Problem

- Each of k actions has an expected reward: value of action
 - ▶ A_t : action selected at time step t
 - ▶ R_t : reward at time step t

- Value of action a :

$$q_*(a) := \mathbb{E}[R_t | A_t = a]$$

- $Q_t(a)$: estimated value of action a at time step t

Action-value Methods

- Natural way to estimate expected reward $q_*(a) \rightarrow$ sample-average

$$\begin{aligned} Q_t(a) &:= \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} \\ &= \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}} \end{aligned}$$

- May instead define $Q_t(a)$ as some default value

Exploitation vs. Exploration

- Exploit: maximize reward given what is known
 - ▶ Take $A_t = \arg \max_a Q_t(a)$
- Explore: collect more information for (potentially) higher reward

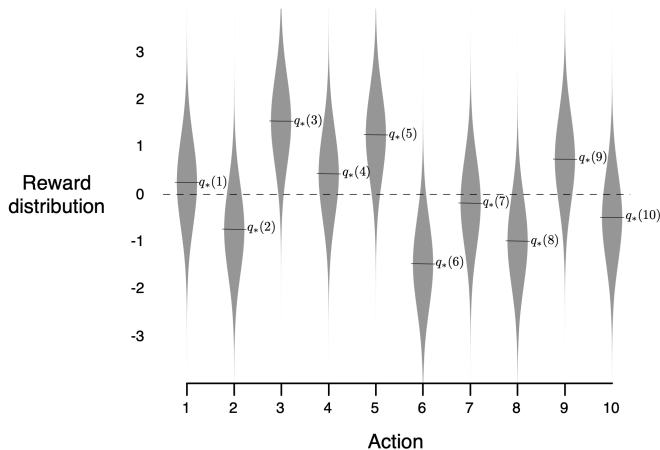
Greedy vs. ϵ -Greedy

- Greedy action selection (exploitation)

$$A_t = \arg \max_a Q_t(a)$$

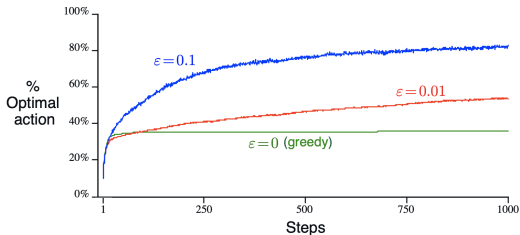
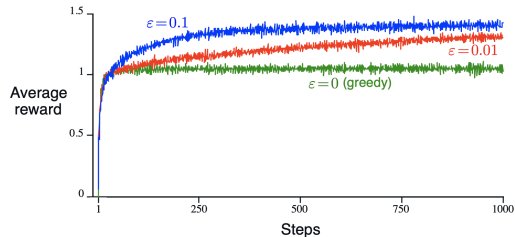
- ϵ -greedy action selection
 - ▶ With probability $1 - \epsilon$, take greedy action $A_t = \arg \max_a Q_t(a)$
 - ▶ With probability ϵ , select randomly among all actions
 - ▶ Advantage: (in the limit) as t increases, every action is sampled an infinite number of times

10-armed Testbed



- Recall: $q_*(a)$ is a **true** expected reward of arm a and $Q_t(a)$ is its estimate at time t
- These distributions and $q_*(a)$ for each arm are unknown to the decision maker.

Performances of ϵ -Greedy and Greedy Policies



- $\epsilon = 0.01$ method improves more slowly, but eventually would perform better than $\epsilon = 0.1$
- It is possible to reduce ϵ over time.

- Noisier rewards (high variance): takes more exploration to find the optimal action
- Question: would ϵ -greedy perform better compared to a greedy policy with noisier rewards?
- What about deterministic rewards (no noise)?

Incremental Implementation

- Q_n : estimate of action value after selected $n - 1$ times

$$Q_n := \frac{R_1 + R_2 + \dots + R_{n-1}}{n - 1}$$

- Memory and computational requirements increase linearly as time increases!
- Alternative: incremental updates

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n]$$

- ▶ Only requires constant memory and computation per time step
- ▶ $\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \cdot [\text{Target} - \text{OldEstimate}]$
- ▶ $[\text{Target} - \text{OldEstimate}]$: error in the action-value estimate

Simple Bandit Algorithm

- Incremental sample-average update + ϵ -greedy action selection

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Loop forever:

$$A \leftarrow \begin{cases} \operatorname{argmax}_a Q(a) & \text{with probability } 1 - \epsilon \quad (\text{breaking ties randomly}) \\ \text{a random action} & \text{with probability } \epsilon \end{cases}$$

$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

Step Size and Convergence

- Incremental update rule with step size α_n

$$Q_{n+1}(a) := Q_n(a) + \alpha_n(a) [R_n(a) - Q_n(a)]$$

- Conditions required to assure convergence (with probability 1)

$$(i) \sum_{n=1}^{\infty} \alpha_n(a) = \infty \quad \text{and} \quad (ii) \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

- ▶ Condition (i): steps should be large enough to eventually overcome any initial condition (or random fluctuations)
- ▶ Condition (ii): steps eventually become small enough to assure convergence

Optimistic Initial Values

- Finite estimates of action-value depend on initial action value $Q_1(a)$.
- Can be used as a simple way to encourage exploration
 - ▶ Can set initial values to be “optimistic” (higher than achievable values)

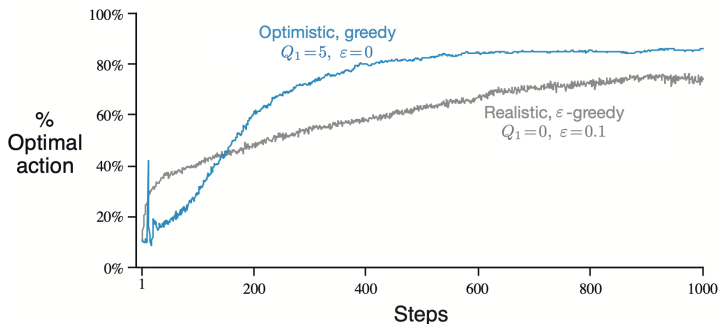


Figure: The effect of optimistic initial action-value estimates on the 10-armed testbed (with constant step-size $\alpha = 0.1$)

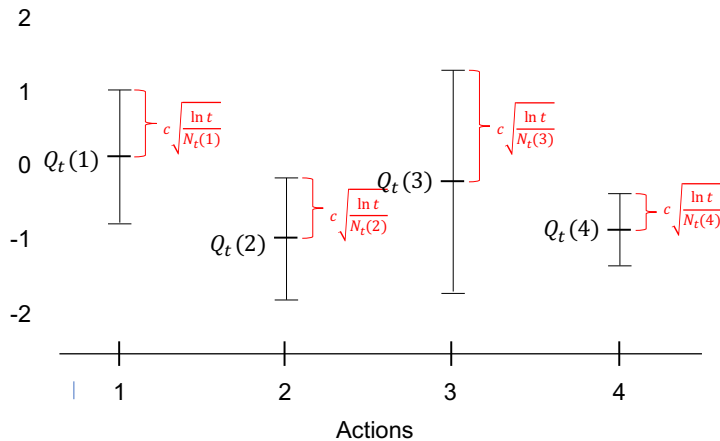
Upper-Confidence-Bound (UCB) Action Selection

- Wouldn't it be better to select among non-greedy actions according to their potential for actually being optimal?
 - ▶ ϵ -greedy action selection ignores this aspect!
 - ▶ Incorporate both how close their estimates are to being optimal and uncertainties in those estimates
- Upper-confidence-bound (UCB) action

$$A_t := \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

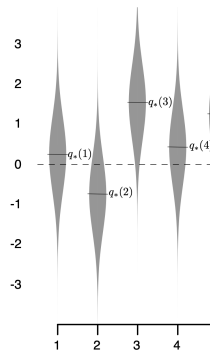
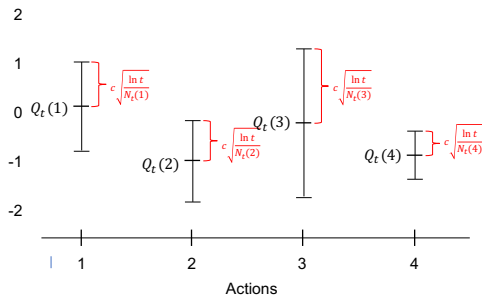
- ▶ $N_t(a)$: the number of times action a has been selected before t
- ▶ c : controls the degree of exploration

UCB Algorithm Illustration

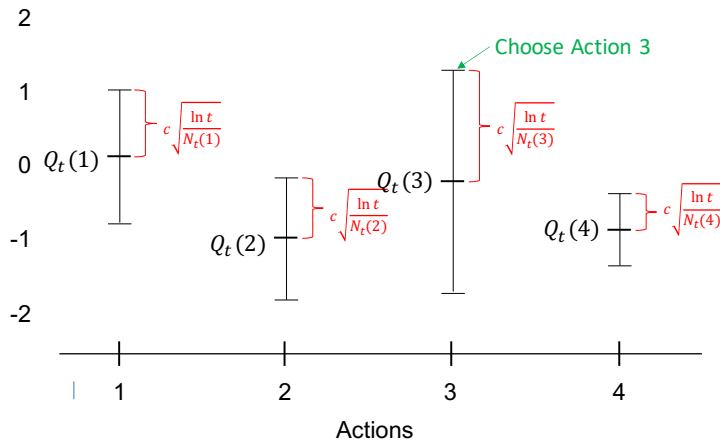


UCB Algorithm Illustration

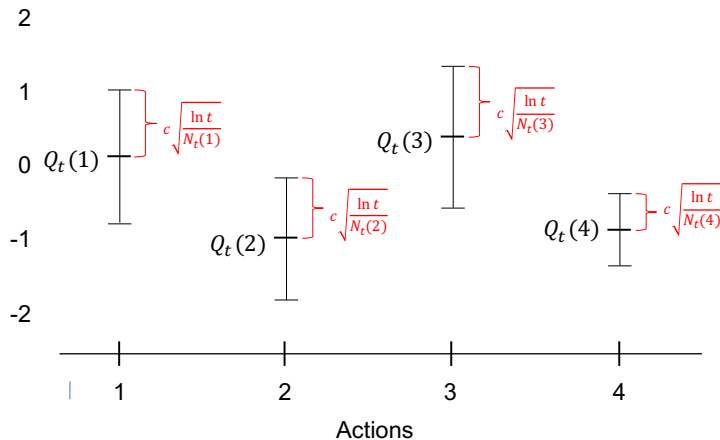
- Note $Q_t(a)$ (estimate) and $q_*(a)$ (true mean) are different



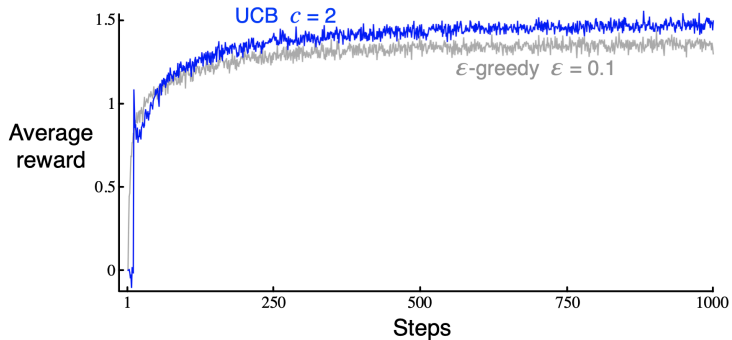
UCB Algorithm Illustration



UCB Algorithm Illustration



Performance of a UCB Algorithm



Gradient Bandit Algorithms

- Learn a numerical **preference** $H_t(a) \in \mathbb{R}$ for each action a
 - ▶ larger $H_t(a) \Rightarrow$ action a is selected more often
 - ▶ has no interpretation in terms of reward

Gradient Bandit Algorithms

- Learn a numerical **preference** $H_t(a) \in \mathbb{R}$ for each action a
 - ▶ larger $H_t(a) \Rightarrow$ action a is selected more often
 - ▶ has no interpretation in terms of reward
- Action selection based on soft-max (Boltzmann) distribution

$$\mathbb{P}\{A_t = a\} := \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}} := \pi_t(a)$$

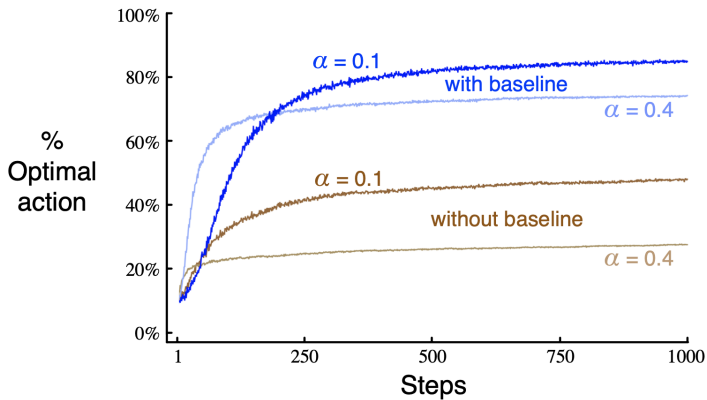
- ▶ Update rule after selecting action A_t and receiving the reward R_t

$$H_{t+1}(A_t) := H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)), \text{ and}$$

$$H_{t+1}(a) := H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a) \quad \forall a \neq A_t$$

where $\bar{R}_t$ is the average of the rewards prior to t .

Performances of Gradient Bandit Algorithms



Non-stationary Rewards

- What happens when rewards of arms are changing over time?

Non-stationary Rewards

- What happens when rewards of arms are changing over time?
- For stationary rewards, α_n (typically) **decreasing** function of n

$$Q_{n+1}(a) := Q_n(a) + \alpha_n(a) [R_n(a) - Q_n(a)]$$

Non-stationary Rewards

- What happens when rewards of arms are changing over time?
- For stationary rewards, α_n (typically) **decreasing** function of n

$$Q_{n+1}(a) := Q_n(a) + \alpha_n(a) [R_n(a) - Q_n(a)]$$

- For non-stationary rewards, use constant step size α

$$Q_{n+1}(a) := Q_n(a) + \alpha [R_n(a) - Q_n(a)]$$

Possible Non-Stationarity?

Question: given the previous sequence of rewards, which arm would you take next?

- Arm 1: 1, 2, 0, 3, 1, 1, 2, 0
- Arm 2: 50, 30, 0, 0, 0, 0, 0, 0

Next class: Markov Decision Processes (Chapter 3 of Sutton & Barto)